

Big Data Coursework Part 2

Report

tomasz.kaczmarski@city.ac.uk

Problem Statement

Doing big data analytics is a challenging task. Measuring success can be done from many perspectives, which include accuracy and performance learning speed among others. While doing data engineering, we can often face problems with high CPU or memory consumption and, as a result, experience a slowdown in data processing and learning or even crashes on servers.

In recent years, we can notice a significant growth in multiple cloud solutions; all the technology giants, such as Amazon, Google and Microsoft, offer their own products in this area. Finding the right cloud solution can help with CPU and RAM memory disk storage utilisation, and others. Very often, finding a balance between price and efficiency becomes crucial for businesses to be competitive.

Also, finding the best configuration for specific data pipelines can be a very challenging task. From my experience, I know an example whereby a particular ETL process was working well on one azure cloud configuration, but on another, the data was not loading at all. As a result, business was significantly affected as the right information was not being received on time.

Techniques

There are multiple approaches to finding the most effective configuration for data flows. One of the most interesting ones is called CherryPick. It is a system that unearths the optimal or near-optimal cloud configurations that minimize cloud usage costs, guarantee application performance and limit the search overhead for recurring big data analytic jobs [1] (cherrypick.pdf, 469). CherryPick uses Bayesian optimization (BO) which means it does not have a predefined model and can run on a small amount of data. It estimates a confidence interval of the cost and running time for each candidate cloud configuration to arrive at a 'good enough' solution

Another approach when working with machine learning models is modifying parameters. The most popular method when dealing with big data problems is to increase the learning rate. On the other hand, great results can be also achieved by modifying batch size or applying hybrid solutions using both.

Both techniques tackle problems related to improving the data flow performance through configuration settings and batch size modification by limiting data volume. This is different from CherryPick.

Part 1 Optimisation techniques and coursework tasks analysis

In previous tasks from the coursework I have been working on, the dataset contained images of flowers. Pictures were divided into five classes based on flower names (roses, dandelions, etc.). The objective of the earlier tasks was to apply different reprocessing methods and machine learning to see how they could be improved

In the first task, the object was to measure the reading speed for different numbers of images by balancing the number of batches, batch size and repetitions. As shown in Figure 1, when running the code in Google Colaboratory, after increasing the batch size, the values were between 30 to 40 images per second when the batch was greater than 20. When running the same code but only increasing the number of images in a single batch (Figure 2), we can see that we can read more images – but only five more per second in the highest number of images.

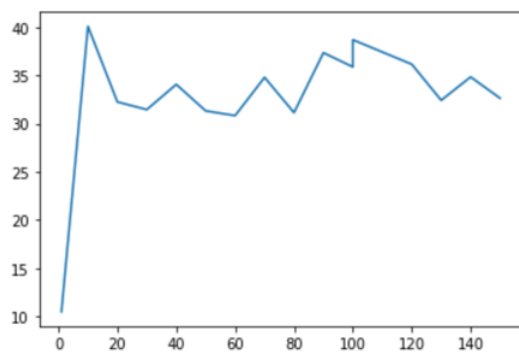


FIGURE 1

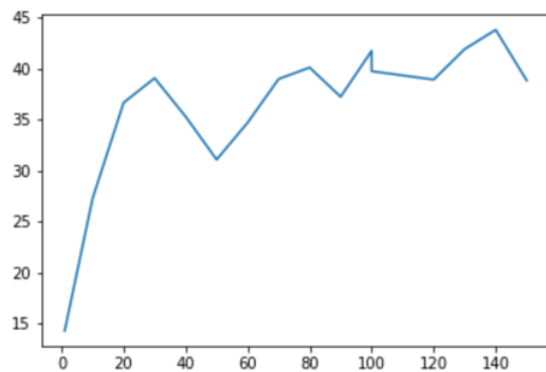


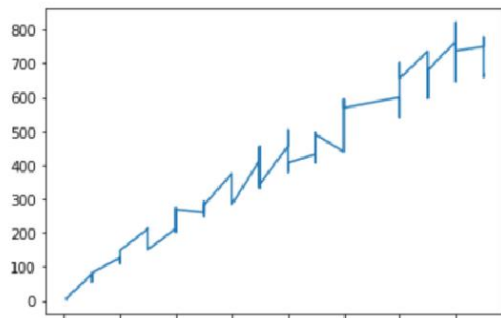
FIGURE 2

To execute the same code with more than 10 batch sizes with 10 batch numbers was not possible because the code was constantly crashing. Increasing the number of repetitions made no difference.

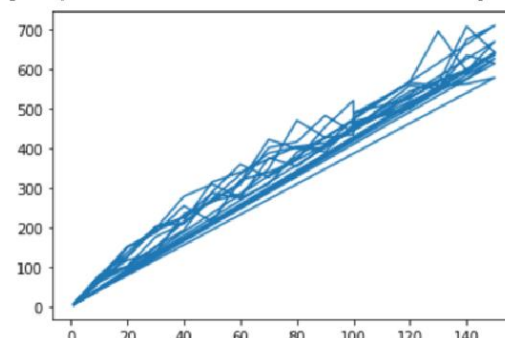
This was not very successful and extremely time-consuming as thousands of images often had to be loaded.

On the other hand, having a high number of images in multiple batches is essential for high levels of accuracy while using machine learning. The image number could be verified using the central limit theory, as well.

In the second task, I applied the codes from the first task, but this time using Google Cloud. Running the code on Cloud allowed me to improve the reading speed by not modifying the dataset but by increasing the number of the CPU, memory. What is more, I was able to run the job in parallel on multiple workers. The results can be found below in the line charts:



TFR & batch number figure 3



TFR & Batch Size Figure 4

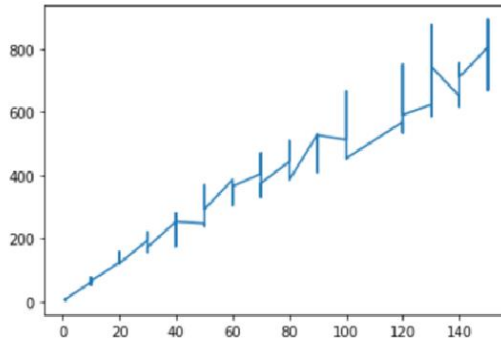


Image and batch number Figure 5

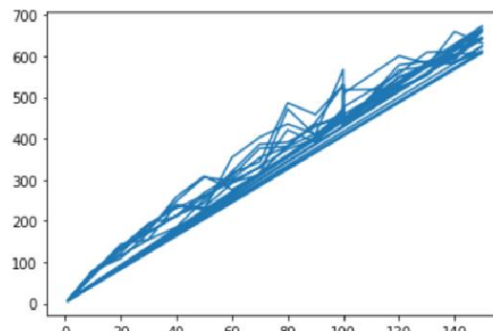


Image and Batch Size Figure 6

When we compare these with the chart from task one, we can immediately see that the number of images per second has increased significantly. However, it is challenging to identify the optimum configuration and get the greatest speed on Cloud, especially as it can be driven by multiple factors, like changing the batch size or number, which will require different configurations. This is where the cherry-picking technique could be applied, after identifying the best number of images. While running the pre-processing in Cloud, we can identify the optimum configuration using the Bayesian algorithm, which would be a good solution. It would allow us to save time while doing multiple experiments, and also the budget as we would be using Google Cloud. What does not look correct is the number of images. The TFR records are similar in reading speed. The TFR was expected to be faster in the coding part but an improvement was not found. However, due to time limitations, I cannot review it.

After pre-processing, we can apply machine learning in Cloud. This is the moment when we can apply the second technique. Modifying the learning rate is a simple parameter adjustment and can be done quickly. The real challenge is to find a good balance between the rate of change and the batch size. I think a very interesting experiment could be to use machine learning to work on a hybrid solution between changing the learning rate and batch size.

After collecting a good number of records, cherry-picking is applied to identify the optimal processing configuration. It is important not to apply any changes to the dataset size after cherry-picking as this might require a different configuration. I believe it would be a very interesting experiment.

PART II Optimisation techniques in different processing strategies

In today's world, digital companies have to deal with giga and terabytes, or even petabytes, of data regularly. It also very important to present data which is as real as possible to stay ahead of the competition. As presented below, there are three main data processing types:

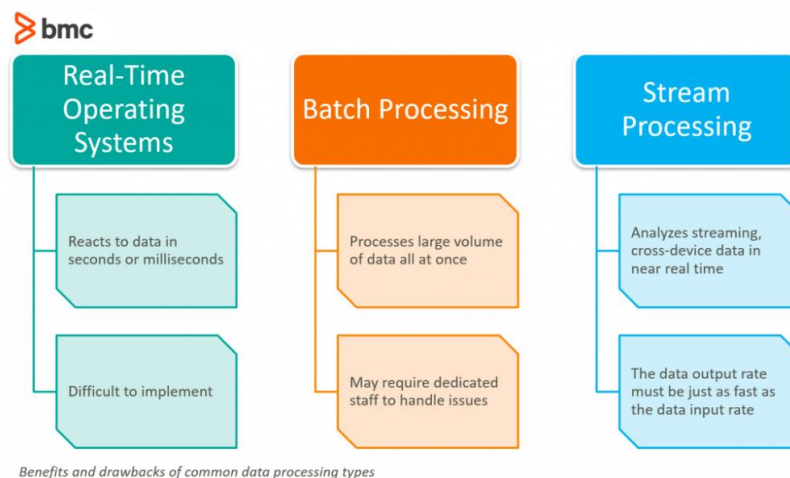


FIGURE 10

Real-Time Operating Systems

This kind of dataflow can be defined briefly as a system which presents data as close as possible to the event which has generated it, often in less than a second. A good example of this can be found in online banking, where taking money from an ATM is being recorded immediately on a balance sheet. Another system where this process occurs is in online trading.

When decaying learning rate we need to be aware that data is processed in small batches constantly (not micro like online) but very tiny this means manipulating learning rate will be less beneficial, however still applicable that kind of system requires high CPU, fast internet to process data rapidly on the other hand less storage, tiny batches

This is a strategy where a cherry-picking technique would be highly beneficial. While processing data regularly, applying correct configuration through machine learning can offer better options on how to set up parameters especially in real-time operating systems where data is constantly growing. This means we need to handle workload changes. CherryPick needs a good selection of representative workloads and we should choose a relatively larger workload as the representative workload (e.g., choosing 125% gives you more stability than choosing 100%). What is more, we can also run cherry-picking programmes regularly to monitor if the parameters are optimal at all times. In this case, it can not only lead to great savings in hardware but also human resources.

Batch processing

In this kind of system, we process data to visualise it in larger groups for the sake of records. Very often, this depends on the business and how frequently the data needs to be refreshed. An example of this can be an accounting system where the values of the main KPI of a company need to be evaluated and presented at the end of each month. Another example can be a warehouse

inventory system, where data needs to be refreshed overnight to present stock quantities from the previous day.

This is a key strategy where we can apply a decaying learning rate technique and manipulate the batch size, learning rate and momentum. The main risk is to make sure that accuracy does not drop below the target-keeping accuracy. If under control, we can dramatically reduce the training times

When applying cherrypicking for offline data processing we need to be aware that processed data is usually large batch size this and in current world of multiple parameters some and in practice setting parameters to maximum point might would not so beneficial that's why decaying learning rate might bring large benefits

Data Streaming

Data streaming is a third processing strategy. In this case, the data is transferred immediately from one device to another and it does not require to be stored. One example of this can be fraud detection. For example, if a transaction looks suspicious, it will be blocked immediately before any fraud can be committed. The transaction must be as fast as possible; this will avoid any issues with storage and memory.

This method can be referred as micro-batch processing in result we can notice high CPU usage, multiple workers used in parallel and low storage

In this case, cherry-picking is an important technique. Data streaming very often requires multiple node CPUs but the data is not stored as it is transmitted immediately. Cherry-picking would be a powerful technique not only for the initial configuration but also for any regular configuration reviews. As its almost labour-free, analyses can even run daily to spot if the system has a good enough configuration and it can create alerts if changes need to be applied.

That a decaying learning rate and increasing batch size operate here in the same way as in real-time operating systems is not much applicable as data need to be processed in tiny batches as fast as possible

On the other hand, if we need a system which is considered to be as close as possible to generating information as a machine learning algorithm, we can consider applying it. For example, in cases of fraud detection, as mentioned earlier.

Summary

There are two main techniques for improving data processing: both have their advantages and disadvantages. What is more not always considered is where they can be applied. On the other hand, selecting an appropriate processing optimisation technique will bring many benefits such as saving CPU storage workers and achieving high data accuracy with a reduction in training times. As a result, this can lead to additional cost-saving for the company and make the business more competitive in the market. If we apply both techniques, first we should reduce the learning rate to get optimum data size, and then apply cherry-picking to get the optimum configuration. Doing it the opposite way around will result in the cherry-picking failing to achieve an accurate volume and the performance will be lower. Also, after the initial implementation, both techniques should be reviewed on a regular basis as data can vary, as well. I think greatest benefit would be in large

companies, such as banks, and less beneficial in tiny start-ups, where the dataset is often varied or processing can only be re-implemented from time to time. What found setting up cluster often mean trading number of nodes for computing power of the individual nodes

REFERENCES

FIGURE 10 - <https://www.bmc.com/blogs/batch-processing-stream-processing-real-time/>

Word Count	?	×
Statistics:		
Pages	6	
Words	1,923	
Characters (no spaces)	9,815	
Characters (with spaces)	11,753	
Paragraphs	42	
Lines	165	
☒ Include textboxes, footnotes and endnotes		
Close		